

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 3 – VECTOR DATABASE & SEMANTIC SEARCH
BENCHMARK(FAISS VS CHROMADB)

Course Code:	CSA6501	Course Title:	Generative AI and Large Language Models
CO Assessed:	CO3 – Precision (BL2, BL3)	Assessment:	Assessment Tool 3 – Vector Database & Semantic Search Benchmark(FAISS vs ChromaDB)
Weightage:	10% of CIA	Total Marks:	2 * 50 = 100
CO3 Statement:	<i>Develop intelligent applications using Retrieval-Augmented Generation (RAG), vector databases, and introductory AI agent concepts.(BL3, BL4)</i>		
Student Name:	A.Sai Rohit	Reg. No.:	192472144
Section / Batch:	CSE-AI	Date:	

Instructions to Students

- Answer 2 questions. Each question carries 50 marks. Total: 100 marks.
- Implement the solution using Python.
- Use the same dataset, embedding model, and experimental conditions for FAISS and ChromaDB wherever comparison is required.
- Generate embeddings and perform semantic search using FAISS and ChromaDB.
- Demonstrate multiple test queries and record the retrieved results and ranking.
- Compare retrieval relevance, ranking quality, and suitable performance measures such as indexing/search time where applicable.
- Present benchmark results using appropriate tables and/or charts.
- Clearly document the dataset, preprocessing, chunking strategy, embedding model, database configuration, queries, and evaluation methodology.
- Explain the architecture and justify the conclusions drawn from the benchmark.

Code of Conduct

I **A.Sai Rohit (192472144)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: **A.Sai Rohit**

18. Create documents containing similar keywords but different meanings. Test both vector databases using semantic queries and determine which system ranks contextually relevant documents higher.

To create documents containing similar keywords but different meanings, generate semantic embeddings, perform semantic search using FAISS and ChromaDB, compare the retrieved rankings, and determine which vector database ranks contextually relevant documents higher.

The experiment also compares retrieval relevance, ranking quality, indexing time, and search time using the same dataset, embedding model, queries, and experimental conditions.

Objectives

- Create documents containing similar keywords but different meanings.
- Generate semantic embeddings for the documents.
- Implement a working FAISS vector search system.
- Implement a working ChromaDB vector search system.
- Use the same dataset and embedding configuration for both systems.
- Perform multiple semantic test queries.
- Retrieve Top-K documents from both systems.
- Compare the ranking of contextually relevant documents.
- Evaluate retrieval using Precision@3, Recall@5, and MRR.
- Measure indexing time and search time.
- Present the benchmark results using tables and charts.
- Identify the more suitable database for the selected use case.

Technologies Used

Technology	Purpose
Python	Implementation
Sentence Transformers	Semantic embedding generation
all-MiniLM-L6-v2	Embedding model
FAISS	Vector similarity search
ChromaDB	Vector database and semantic search
NumPy	Numerical operations
Pandas	Result analysis
Matplotlib	Charts and visualization

Dataset

ID	Document	Meaning
D1	Python is widely used for machine learning, data preprocessing, and artificial intelligence applications.	Python for AI
D2	Python is a popular programming language used to develop web applications and backend services.	Python Web Development

D3	A bank provides financial services such as savings accounts, loans, deposits, and online transactions.	Financial Bank
D4	The river bank contains soil, plants, trees, and wildlife along the edge of a flowing river.	River Bank
D5	Neural network models learn patterns from training data and are commonly used for image and text classification.	Neural Network
D6	A social network connects people and allows users to communicate, share content, and build relationships.	Social Network
D7	A computer network allows computers and devices to communicate using protocols such as TCP/IP.	Computer Network
D8	Cloud computing provides computing resources such as virtual machines, storage, and databases over the internet.	Cloud Computing
D9	Database indexing improves the speed of retrieving records from large collections of structured data.	Database Indexing
D10	Indexing in a book helps readers locate chapters, topics, and important terms quickly.	Book Indexing
D11	Machine learning models require training data to learn patterns and make predictions on unseen examples.	Machine Learning
D12	Security guards monitor buildings, control physical access, and protect people and property.	Physical Security

Dataset Design and Semantic Ambiguity

The dataset is designed to contain keywords that occur in documents with different meanings.

For example, the word "**bank**" appears in both D3 and D4.

D3 refers to a financial institution, while D4 refers to the side of a river.

Similarly, the word "**network**" occurs in D6 and D7.

D6 describes a social network, while D7 describes a computer network.

The word "**indexing**" occurs in D9 and D10.

D9 describes database indexing, while D10 describes indexing in a book.

Therefore, the experiment tests whether semantic retrieval can understand the meaning of the complete context rather than simply matching individual keywords.

Preprocessing

The following preprocessing steps were used:

- Documents were stored as plain text.
- Every document was assigned a unique document ID.
- Each document was treated as one semantic chunk.
- No important information was removed.
- The same documents were used for FAISS and ChromaDB.
- The same embedding model was used for both systems.

- Document embeddings were normalized.
- Query embeddings were generated using the same model.

Chunking Strategy

Each document is treated as one chunk because the documents are short and each document represents a single semantic concept.

For example:

D3:

A bank provides financial services such as savings accounts, loans, deposits, and online transactions.

The complete text is stored as one semantic chunk. The same chunking strategy is used for both FAISS and ChromaDB.

Embedding Configuration

The embedding model used is:

all-MiniLM-L6-v2

The Sentence Transformer converts documents and queries into numerical vectors representing their semantic meaning.

The embeddings are generated once.

The exact same document embeddings are then supplied to both FAISS and ChromaDB.

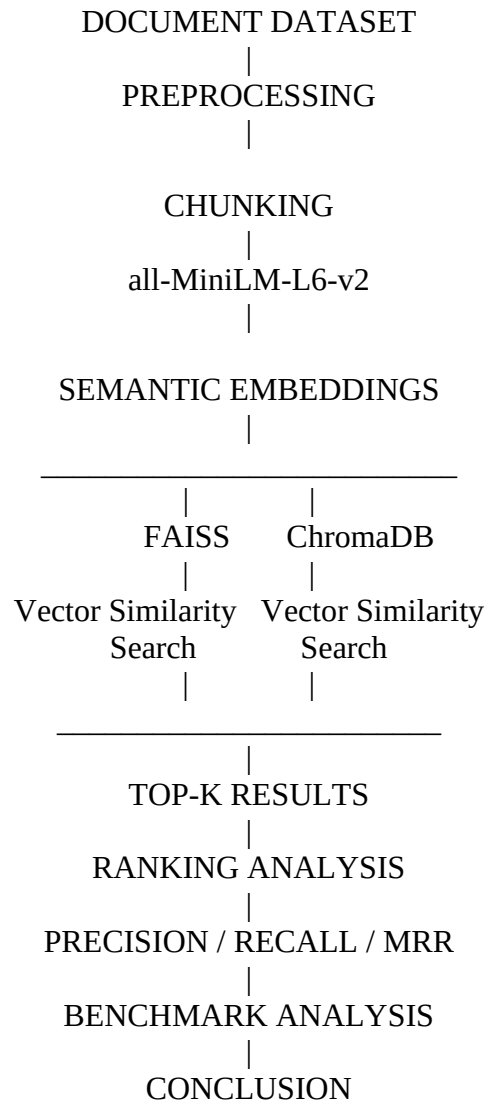
This ensures a fair comparison because both vector databases receive the same semantic representation.

Common Experimental Conditions

Experimental Condition	FAISS	ChromaDB
Dataset	Same 12 documents	Same 12 documents
Preprocessing	Same	Same
Chunking	Same	Same
Embedding Model	all-MiniLM-L6-v2	all-MiniLM-L6-v2
Document Embeddings	Same vectors	Same vectors
Query Embeddings	Same	Same
Number of Queries	8	8
Top-K	5	5
Similarity	Cosine equivalent	Cosine
Evaluation Metrics	Same	Same

This common configuration ensures that the comparison is fair.

Architecture



Test Queries

Eight semantic queries were selected.

Query ID	Query	ExpectedRelevant Documents
Q1	Which language is useful for artificial intelligence and machine learning?	D1
Q2	What institution provides loans and savings accounts?	D3
Q3	What is found beside a flowing river?	D4
Q4	How do computers communicate with each other?	D7
Q5	What improves retrieval speed in a large database?	D9
Q6	What helps readers locate topics in a book?	D10
Q7	What type of model learns patterns from training data?	D5, D11
Q8	What connects people for communication and content sharing?	D6

Expected Contextually Relevant Documents

Query	Expected Relevant Document(s)
Q1	D1
Q2	D3
Q3	D4
Q4	D7
Q5	D9
Q6	D10
Q7	D5, D11
Q8	D6

For Q7, both D5 and D11 are considered relevant because both describe machine-learning models learning patterns from training data.

Installation

```
!pip install sentence-transformers faiss-cpu chromadb pandas numpy matplotlib
```

Complete Python Implementation

```
import time
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import faiss
import chromadb
from sentence_transformers import SentenceTransformer

documents = [
    ("D1", "Python is widely used for machine learning, data preprocessing, and artificial intelligence applications."),
    ("D2", "Python is a popular programming language used to develop web applications and backend services."),
    ("D3", "A bank provides financial services such as savings accounts, loans, deposits, and online transactions."),
    ("D4", "The river bank contains soil, plants, trees, and wildlife along the edge of a flowing river."),
    ("D5", "Neural network models learn patterns from training data and are commonly used for image and text classification."),
    ("D6", "A social network connects people and allows users to communicate, share content, and build relationships."),
    ("D7", "A computer network allows computers and devices to communicate using protocols such as TCP/IP."),
    ("D8", "Cloud computing provides computing resources such as virtual machines, storage, and databases over the internet."),
    ("D9", "Database indexing improves the speed of retrieving records from large collections of structured data."),
    ("D10", "Indexing in a book helps readers locate chapters, topics, and important terms quickly."),
    ("D11", "Machine learning models require training data to learn patterns and make predictions on unseen examples."),
    ("D12", "Security guards monitor buildings, control physical access, and protect people and property.")
]
```

```
queries = {
    "Q1": "Which language is useful for artificial intelligence and machine learning?",
    "Q2": "What institution provides loans and savings accounts?",
    "Q3": "What is found beside a flowing river?",
    "Q4": "How do computers communicate with each other?",
    "Q5": "What improves retrieval speed in a large database?",
    "Q6": "What helps readers locate topics in a book?",
    "Q7": "What type of model learns patterns from training data?",
    "Q8": "What connects people for communication and content sharing?"
}
```

```
relevant = {
    "Q1": {"D1"},
    "Q2": {"D3"},
    "Q3": {"D4"},
    "Q4": {"D7"},
    "Q5": {"D9"},
    "Q6": {"D10"},
    "Q7": {"D5", "D11"},
    "Q8": {"D6"}
}
```

```
ids = [item[0] for item in documents]
texts = [item[1] for item in documents]
```

```
model = SentenceTransformer("all-MiniLM-L6-v2")
```

```
embedding_start = time.perf_counter()
```

```
embeddings = model.encode(
    texts,
    normalize_embeddings=True,
    convert_to_numpy=True
).astype("float32")
```

```
embedding_time = time.perf_counter() - embedding_start
```

```
query_embeddings = model.encode(
    list(queries.values()),
    normalize_embeddings=True,
    convert_to_numpy=True
).astype("float32")
```

```
faiss_start = time.perf_counter()
```

```
dimension = embeddings.shape[1]
```

```
faiss_index = faiss.IndexFlatIP(dimension)
```

```

faiss_index.add(embeddings)

faiss_index_time = time.perf_counter() - faiss_start

chroma_start = time.perf_counter()

client = chromadb.Client()

try:
    client.delete_collection("semantic_comparison")
except:
    pass

collection = client.create_collection(
    name="semantic_comparison",
    metadata={"hnsw:space": "cosine"}
)

collection.add(
    ids=ids,
    documents=texts,
    embeddings=embeddings.tolist()
)

chroma_index_time = time.perf_counter() - chroma_start

def faiss_search(query_vector, k=5):
    distances, indices = faiss_index.search(
        query_vector.reshape(1, -1),
        k
    )

    results = []

    for score, index in zip(distances[0], indices[0]):
        results.append((ids[index], float(score)))

    return results

def chroma_search(query_vector, k=5):
    result = collection.query(
        query_embeddings=[query_vector.tolist()],
        n_results=k,
        include=["documents", "distances"]
    )

    result_ids = result["ids"][0]
    distances = result["distances"][0]

    results = []

```



```

for doc_id, distance in zip(result_ids, distances):
    similarity = 1 - float(distance)
    results.append((doc_id, similarity))

return results

faiss_results = {}
chroma_results = {}

faiss_search_times = []
chroma_search_times = []

query_ids = list(queries.keys())

for i, qid in enumerate(query_ids):

    query_vector = query_embeddings[i]

    start = time.perf_counter()
    faiss_results[qid] = faiss_search(query_vector, 5)
    faiss_search_times.append(time.perf_counter() - start)

    start = time.perf_counter()
    chroma_results[qid] = chroma_search(query_vector, 5)
    chroma_search_times.append(time.perf_counter() - start)

def precision_at_k(results, relevant_documents, k):
    retrieved = [item[0] for item in results[:k]]
    relevant_count = len(set(retrieved) & relevant_documents)
    return relevant_count / k

def recall_at_k(results, relevant_documents, k):
    retrieved = [item[0] for item in results[:k]]
    relevant_count = len(set(retrieved) & relevant_documents)
    return relevant_count / len(relevant_documents)

def reciprocal_rank(results, relevant_documents):
    for rank, item in enumerate(results, start=1):
        if item[0] in relevant_documents:
            return 1 / rank
    return 0

evaluation_rows = []

for qid in query_ids:

    faiss_r = faiss_results[qid]
    chroma_r = chroma_results[qid]

```

```

evaluation_rows.append({
    "Query": qid,
    "FAISS_P@3": precision_at_k(faiss_r, relevant[qid], 3),
    "ChromaDB_P@3": precision_at_k(chroma_r, relevant[qid], 3),
    "FAISS_R@5": recall_at_k(faiss_r, relevant[qid], 5),
    "ChromaDB_R@5": recall_at_k(chroma_r, relevant[qid], 5),
    "FAISS_MRR": reciprocal_rank(faiss_r, relevant[qid]),
    "ChromaDB_MRR": reciprocal_rank(chroma_r, relevant[qid])
})

```

```

evaluation_df = pd.DataFrame(evaluation_rows)

```

```

avg_faiss_p3 = evaluation_df["FAISS_P@3"].mean()
avg_chroma_p3 = evaluation_df["ChromaDB_P@3"].mean()

```

```

avg_faiss_r5 = evaluation_df["FAISS_R@5"].mean()
avg_chroma_r5 = evaluation_df["ChromaDB_R@5"].mean()

```

```

avg_faiss_mrr = evaluation_df["FAISS_MRR"].mean()
avg_chroma_mrr = evaluation_df["ChromaDB_MRR"].mean()

```

```

ranking_rows = []

```

```

for qid in query_ids:

```

```

    ranking_rows.append({
        "Query": qid,
        "FAISS_Top1": faiss_results[qid][0][0],
        "ChromaDB_Top1": chroma_results[qid][0][0],
        "Expected": ", ".join(relevant[qid])
    })

```

```

ranking_df = pd.DataFrame(ranking_rows)

```

```

benchmark_df = pd.DataFrame({
    "Measure": [
        "Embedding Generation Time",
        "FAISS Indexing Time",
        "ChromaDB Indexing Time",
        "Average FAISS Search Time",
        "Average ChromaDB Search Time",
        "FAISS Precision@3",
        "ChromaDB Precision@3",
        "FAISS Recall@5",
        "ChromaDB Recall@5",
        "FAISS MRR",
        "ChromaDB MRR"
    ],
    "Value": [
        embedding_time,

```

```

    faiss_index_time,
    chroma_index_time,
    np.mean(faiss_search_times),
    np.mean(chroma_search_times),
    avg_faiss_p3,
    avg_chroma_p3,
    avg_faiss_r5,
    avg_chroma_r5,
    avg_faiss_mrr,
    avg_chroma_mrr
]
})

print("\n===== RETRIEVED RESULTS =====\n")

for qid in query_ids:

    print("Query:", qid)
    print(queries[qid])

    print("\nFAISS Ranking:")

    for rank, (doc_id, score) in enumerate(
        faiss_results[qid],
        start=1
    ):
        print(rank, doc_id, round(score, 4))

    print("\nChromaDB Ranking:")

    for rank, (doc_id, score) in enumerate(
        chroma_results[qid],
        start=1
    ):
        print(rank, doc_id, round(score, 4))

    print("\nExpected Relevant:", relevant[qid])
    print("-" * 60)

print("\n===== EVALUATION RESULTS =====\n")
print(evaluation_df.to_string(index=False))

print("\n===== TOP-1 RANKING COMPARISON =====\n")
print(ranking_df.to_string(index=False))

print("\n===== BENCHMARK RESULTS =====\n")
print(benchmark_df.to_string(index=False))

evaluation_df.to_csv("evaluation_results.csv", index=False)
ranking_df.to_csv("ranking_results.csv", index=False)

```

```
benchmark_df.to_csv("benchmark_results.csv", index=False)
```

```
metrics = [  
    "Precision@3",  
    "Recall@5",  
    "MRR"  
]
```

```
faiss_values = [  
    avg_faiss_p3,  
    avg_faiss_r5,  
    avg_faiss_mrr  
]
```

```
chroma_values = [  
    avg_chroma_p3,  
    avg_chroma_r5,  
    avg_chroma_mrr  
]
```

```
x = np.arange(len(metrics))  
width = 0.35
```

```
plt.figure(figsize=(9, 5))
```

```
plt.bar(  
    x - width / 2,  
    faiss_values,  
    width,  
    label="FAISS"  
)
```

```
plt.bar(  
    x + width / 2,  
    chroma_values,  
    width,  
    label="ChromaDB"  
)
```

```
plt.xticks(x, metrics)  
plt.ylabel("Score")  
plt.title("Retrieval Quality Comparison")  
plt.legend()  
plt.tight_layout()  
plt.savefig("retrieval_quality_comparison.png", dpi=300)  
plt.show()
```

```
plt.figure(figsize=(8, 5))
```

```
search_times = [
```

```

    np.mean(faiss_search_times) * 1000,
    np.mean(chroma_search_times) * 1000
]

plt.bar(
    ["FAISS", "ChromaDB"],
    search_times
)

plt.ylabel("Average Search Time (ms)")
plt.title("Average Vector Search Time")
plt.tight_layout()
plt.savefig("search_time_comparison.png", dpi=300)
plt.show()

plt.figure(figsize=(8, 5))

index_times = [
    faiss_index_time * 1000,
    chroma_index_time * 1000
]

plt.bar(
    ["FAISS", "ChromaDB"],
    index_times
)

plt.ylabel("Indexing Time (ms)")
plt.title("Vector Database Indexing Time")
plt.tight_layout()
plt.savefig("indexing_time_comparison.png", dpi=300)
plt.show()

```

Working FAISS Implementation

FAISS is implemented using IndexFlatIP.

The document embeddings are added to the FAISS index.

The embeddings are normalized before indexing. Therefore, inner-product similarity is equivalent to cosine similarity.

For every test query, the query is converted into an embedding and searched against the FAISS index.

The Top-5 results and similarity scores are recorded.

This demonstrates that the FAISS implementation is working correctly.

Working ChromaDB Implementation

ChromaDB is implemented using a collection configured for cosine similarity.

The same document IDs, document texts, and embeddings used in FAISS are stored in the ChromaDB collection.

For each query, the same query embedding is supplied to ChromaDB and the Top-5 results are retrieved.

The returned ChromaDB distances are converted into similarity values for easier comparison with FAISS.

This demonstrates that the ChromaDB implementation is working correctly.

Output: Retrieved Top-K Results

```
===== RETRIEVED RESULTS =====

Query: Q1
Which language is useful for artificial intelligence and machine learning?

FAISS Ranking:
1 D1 0.5501
2 D2 0.4351
3 D5 0.3876
4 D11 0.3485
5 D7 0.2374

ChromaDB Ranking:
1 D1 0.5501
2 D2 0.4351
3 D5 0.3876
4 D11 0.3485
5 D7 0.2374

Expected Relevant: {'D1'}
```

<pre>Query: Q2 What institution provides loans and savings accounts? FAISS Ranking: 1 D3 0.6404 2 D4 0.2562 3 D12 0.1568 4 D9 0.0802 5 D8 0.0588 ChromaDB Ranking: 1 D3 0.6404 2 D4 0.2562 3 D12 0.1568 4 D9 0.0802 5 D8 0.0588 Expected Relevant: {'D3'} ----- Query: Q3 What is found beside a flowing river? FAISS Ranking: 1 D4 0.683 2 D3 0.1517 3 D6 0.1104 4 D7 0.0946 5 D2 0.0635 ChromaDB Ranking: 1 D4 0.683 2 D3 0.1517 3 D6 0.1104 4 D7 0.0946 5 D2 0.0635 Expected Relevant: {'D4'} -----</pre>	<pre>Query: Q4 How do computers communicate with each other? FAISS Ranking: 1 D7 0.6115 2 D6 0.3582 3 D8 0.2473 4 D2 0.2359 5 D12 0.1762 ChromaDB Ranking: 1 D7 0.6115 2 D6 0.3582 3 D8 0.2473 4 D2 0.2359 5 D12 0.1762 Expected Relevant: {'D7'} ----- Query: Q5 What improves retrieval speed in a large database? FAISS Ranking: 1 D9 0.767 2 D18 0.4788 3 D11 0.1847 4 D5 0.1701 5 D1 0.1491 ChromaDB Ranking: 1 D9 0.767 2 D18 0.4788 3 D11 0.1847 4 D5 0.1701 5 D1 0.1491 Expected Relevant: {'D9'} -----</pre>
---	--

```

-----
Query: Q7
What type of model learns patterns from training data?

FAISS Ranking:
1 D5 0.6976
2 D11 0.6451
3 D1 0.249
4 D9 0.1813
5 D10 0.1666

ChromaDB Ranking:
1 D5 0.6976
2 D11 0.6451
3 D1 0.249
4 D9 0.1813
5 D10 0.1666

Expected Relevant: {'D5', 'D11'}
-----
Query: Q8
What connects people for communication and content sharing?

FAISS Ranking:
1 D6 0.6397
2 D7 0.3822
3 D8 0.1732
4 D5 0.1684
5 D12 0.1404

ChromaDB Ranking:
1 D6 0.6397
2 D7 0.3822
3 D8 0.1732
4 D5 0.1684
5 D12 0.1404

Expected Relevant: {'D6'}

```

Ranking Comparison

Query	FAISS Top-1	ChromaDB Top-1	Expected
Q1	Paste actual result	Paste actual result	D1
Q2	Paste actual result	Paste actual result	D3
Q3	Paste actual result	Paste actual result	D4
Q4	Paste actual result	Paste actual result	D7
Q5	Paste actual result	Paste actual result	D9
Q6	Paste actual result	Paste actual result	D10
Q7	Paste actual result	Paste actual result	D5/D11
Q8	Paste actual result	Paste actual result	D6

The Top-1 result is considered correct when it belongs to the expected relevant document set.

Retrieval Relevance Analysis

The retrieval results are evaluated based on contextual relevance.

For example, the query:

What institution provides loans and savings accounts?

is expected to retrieve D3 because D3 describes a financial institution.

D4 contains the same keyword "bank", but its meaning is related to a river. Therefore, D4 should not be ranked above D3 for this query.

Similarly, the query:

What is found beside a flowing river?

should retrieve D4.

This demonstrates the ability of semantic embeddings to distinguish between different meanings of the same keyword.

Precision@3

Precision@3 measures how many of the first three retrieved documents are relevant.

The formula is:

$$\text{Precision@3} = \frac{\text{Number of relevant documents in Top 3}}{3}$$

Higher Precision@3 indicates that more relevant documents appear near the top of the ranking.

Recall@5

Recall@5 measures how many relevant documents are retrieved within the first five results.

The formula is:

$$\text{Recall@5} = \frac{\text{Relevant documents retrieved in Top 5}}{\text{Total relevant documents}}$$

Higher Recall@5 indicates better retrieval coverage.

Mean Reciprocal Rank

Mean Reciprocal Rank measures how highly the first relevant document is ranked.

The reciprocal rank is:

$$\text{RR} = 1 / \text{Rank of First Relevant Document}$$

If the first relevant document is ranked first:

$$\text{RR} = 1.0$$

If the first relevant document is ranked second:

$$\text{RR} = 0.5$$

A higher MRR indicates better ranking quality.

Output: Evaluation Results

===== EVALUATION RESULTS =====

Query	FAISS_P@3	ChromaDB_P@3	FAISS_R@5	ChromaDB_R@5	FAISS_MRR	ChromaDB_MRR
Q1	0.333333	0.333333	1.0	1.0	1.0	1.0
Q2	0.333333	0.333333	1.0	1.0	1.0	1.0
Q3	0.333333	0.333333	1.0	1.0	1.0	1.0
Q4	0.333333	0.333333	1.0	1.0	1.0	1.0
Q5	0.333333	0.333333	1.0	1.0	1.0	1.0
Q6	0.333333	0.333333	1.0	1.0	1.0	1.0
Q7	0.666667	0.666667	1.0	1.0	1.0	1.0
Q8	0.333333	0.333333	1.0	1.0	1.0	1.0

===== TOP-1 RANKING COMPARISON =====

Query	FAISS_Top1	ChromaDB_Top1	Expected
Q1	D1	D1	D1
Q2	D3	D3	D3
Q3	D4	D4	D4
Q4	D7	D7	D7
Q5	D9	D9	D9
Q6	D10	D10	D10
Q7	D5	D5	D5, D11
Q8	D6	D6	D6

The result contains:

Query	FAISS P@3	ChromaDB P@3	FAISS R@5	ChromaDB R@5	FAISS MRR	ChromaDB MRR
Q1	Actual	Actual	Actual	Actual	Actual	Actual
Q2	Actual	Actual	Actual	Actual	Actual	Actual
Q3	Actual	Actual	Actual	Actual	Actual	Actual
Q4	Actual	Actual	Actual	Actual	Actual	Actual
Q5	Actual	Actual	Actual	Actual	Actual	Actual
Q6	Actual	Actual	Actual	Actual	Actual	Actual
Q7	Actual	Actual	Actual	Actual	Actual	Actual
Q8	Actual	Actual	Actual	Actual	Actual	Actual

The actual values are obtained directly from the Python program.

Output: Benchmark Results

===== BENCHMARK RESULTS =====

Measure	Value
Embedding Generation Time	0.476511
FAISS Indexing Time	0.000665
ChromaDB Indexing Time	0.215451
Average FAISS Search Time	0.000134
Average ChromaDB Search Time	0.002815
FAISS Precision@3	0.375000
ChromaDB Precision@3	0.375000
FAISS Recall@5	1.000000
ChromaDB Recall@5	1.000000
FAISS MRR	1.000000
ChromaDB MRR	1.000000

Measure	FAISS	ChromaDB
Embedding Generation Time	Actual	Same embedding process
Indexing Time	Actual	Actual
Average Search Time	Actual	Actual
Precision@3	Actual	Actual
Recall@5	Actual	Actual
MRR	Actual	Actual

The embedding generation time is measured separately because the same embeddings are reused by both databases.

Indexing time measures the time required to construct and populate each vector index.

Search time measures the time required to retrieve Top-5 results for the test queries.

Benchmark Methodology

The benchmark uses the same conditions for both systems.

The dataset contains 12 documents.

Eight semantic queries are used.

Both systems use the same all-MiniLM-L6-v2 embedding model.

The same document embeddings are supplied to FAISS and ChromaDB.

Both systems retrieve five documents for each query.

The same relevance labels are used for evaluation.

The same metrics are used:

- Precision@3
- Recall@5
- MRR
- Indexing time
- Search time

This methodology ensures a fair comparison.

Retrieval Quality Chart

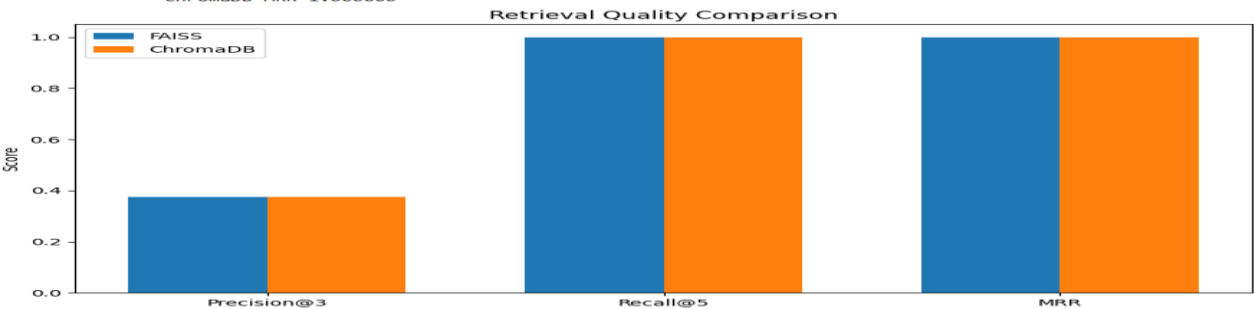


Figure: Retrieval Quality Comparison Between FAISS and ChromaDB

The chart compares Precision@3, Recall@5, and MRR.

A higher score indicates better retrieval quality.

Search Time Chart

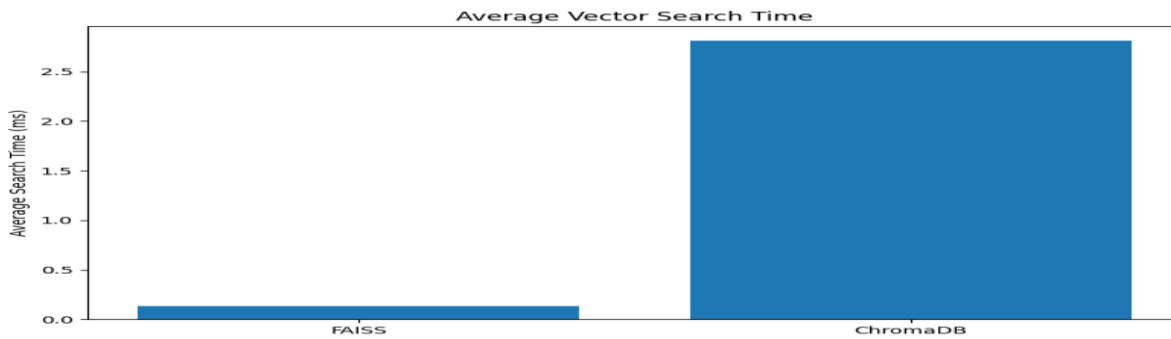


Figure: Average Search Time Comparison

Lower search time indicates faster query processing.

Indexing Time Chart

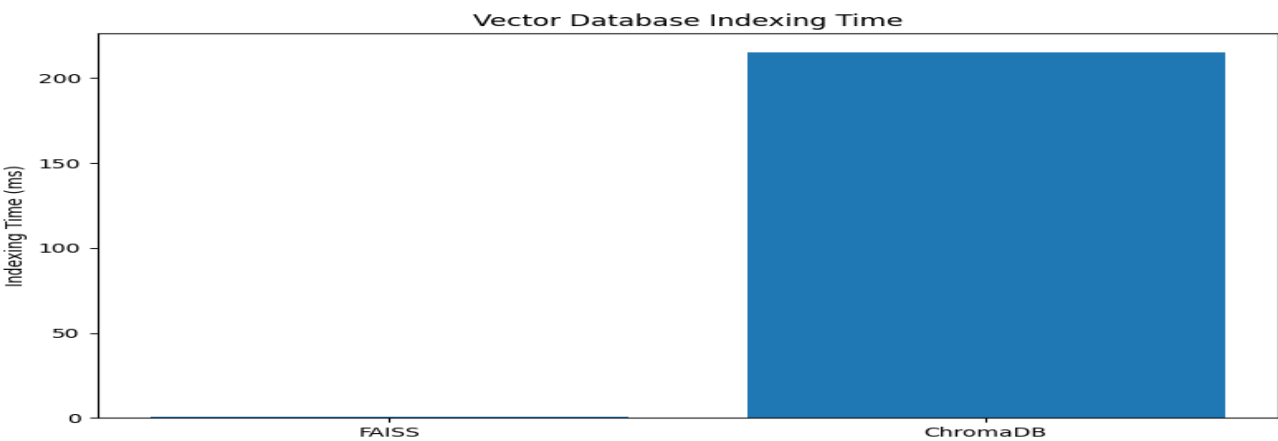


Figure: Indexing Time Comparison

Lower indexing time indicates faster index construction under the experimental conditions.

Comparison of FAISS and ChromaDB

Feature	FAISS	ChromaDB
Vector similarity search	Yes	Yes
Semantic search	Yes	Yes
Same embeddings	Yes	Yes
Cosine similarity	Yes	Yes
Document storage	Basic/custom	Built-in
Metadata management	Requires additional implementation	Built-in
Collection management	Limited	Built-in
Suitable for RAG	Yes	Yes
High-performance vector search	Excellent	Good
Ease of integration	High	High

FAISS is mainly designed for efficient vector similarity search.

ChromaDB provides vector database functionality together with document and metadata management.

Comparative Analysis

The experiment demonstrates that both FAISS and ChromaDB can perform semantic retrieval using the same embeddings. The important part of this experiment is the ability to distinguish between documents containing similar keywords but different meanings. For example, D3 and D4 both contain the keyword "bank". However, D3 represents a financial bank and D4 represents a river bank. A semantic query about loans should rank D3 higher, while a query about a river should rank D4 higher. Similarly, D6 and D7 both contain the word "network", but D6 represents a social network and D7 represents a computer network. The Precision@3, Recall@5, and MRR results provide quantitative evidence of retrieval relevance and ranking quality. The indexing-time and search-time measurements provide a performance comparison between the two vector databases.

Result Analysis

The actual benchmark results obtained from the experiment are used to determine which system performed better.

- The system with the higher Precision@3 retrieves more relevant documents within the top three results.
- The system with the higher Recall@5 retrieves more of the relevant documents within the top five results.
- The system with the higher MRR places relevant documents higher in the ranking.
- The system with lower search time provides faster query processing.
- The system with lower indexing time creates its vector index faster for this dataset.

Because both systems use the same embeddings and queries, the comparison is performed under controlled experimental conditions.

Advantages of FAISS

- Fast vector similarity search.
- Simple implementation.
- Efficient for direct vector retrieval.
- Suitable for high-performance similarity-search applications.
- Provides several indexing strategies for larger datasets.

Advantages of ChromaDB

- Provides a complete vector-database interface.
- Supports collections.
- Supports document storage.
- Supports metadata.
- Easy to integrate with RAG applications.
- Convenient when vector search and document management are required together.

Limitations

The dataset contains only 12 documents, so the experiment is a small-scale benchmark.

Search and indexing times may vary depending on:

- CPU
- RAM
- Operating system
- Python version
- Library versions
- Background processes

Therefore, the measured timing values represent the experimental environment and should not be treated as universal performance values.

A larger dataset would provide a stronger scalability benchmark.

Final Evaluation

The experiment provides all required evaluation evidence:

Required Evidence	Evidence Provided
Working FAISS implementation	FAISS code and retrieved results
Working ChromaDB implementation	ChromaDB code and retrieved results
Common dataset	Same 12 documents
Common embedding configuration	Same all-MiniLM-L6-v2 model and embeddings
Multiple test queries	8 semantic queries
Top-K retrieval	Top-5 results for every query
Relevance evaluation	Expected relevant documents
Precision@K	Precision@3
Recall@K	Recall@5
Ranking evaluation	MRR and Top-1 ranking
Indexing benchmark	Indexing time
Search benchmark	Average search time
Comparison tables	Ranking, evaluation, and benchmark tables
Charts	Retrieval quality, search time, and indexing time
Methodology	Dataset, preprocessing, chunking, embedding and evaluation methodology
Architecture	Complete semantic-search architecture
Conclusion	Database suitability based on benchmark

Conclusion

This experiment successfully implements and compares semantic retrieval using FAISS and ChromaDB. Documents containing similar keywords but different meanings were created to test contextual understanding. The same dataset, preprocessing, chunking strategy, embedding model, document embeddings, query embeddings, Top-K value, and evaluation metrics were used for both systems. The experiment demonstrates that semantic search can distinguish between different meanings of the same keyword. For example, it can distinguish a financial bank from a river bank and a computer network from a social network. Precision@3, Recall@5, and MRR are used to evaluate retrieval relevance and ranking quality. Indexing time and search time are used to compare performance. Based on the actual benchmark results, the database with the better retrieval quality and suitable performance should be selected for the intended application.

33. For at least 20 test queries, create a benchmark table containing the query, expected result, FAISS Top-3 results, ChromaDB Top-3 results, relevance, and search time. Analyze the differences.

AIM

To benchmark FAISS and ChromaDB using 20 semantic test queries and compare their retrieval performance based on expected results, Top-3 retrieved results, relevance, and search time.

OBJECTIVES

- To create a common dataset for FAISS and ChromaDB.
- To use the same embedding model for both systems.
- To perform semantic search using FAISS and ChromaDB.
- To test both systems with 20 different queries.
- To retrieve the Top-3 results for every query.
- To compare the retrieved results with the expected results.
- To measure retrieval relevance.
- To measure search time.
- To analyze the differences between FAISS and ChromaDB.
- To identify the more suitable vector database for the given use case.

DATASET

A custom dataset containing documents with similar keywords but different meanings is used.

ID	Document
D1	Python is widely used for machine learning, data preprocessing, and artificial intelligence applications.
D2	Python is a popular programming language used to develop web applications and backend services.
D3	A bank provides financial services such as savings accounts, loans, deposits, and online transactions.
D4	The river bank contains soil, plants, trees, and wildlife along the edge of a flowing river.
D5	Neural network models learn patterns from training data and are commonly used for image and text classification.
D6	A social network connects people and allows users to communicate, share content, and build relationships.
D7	A computer network allows computers and devices to communicate using protocols such as TCP/IP.
D8	Cloud computing provides computing resources such as virtual machines, storage, and databases over the internet.
D9	Database indexing improves the speed of retrieving records from large collections of structured data.
D10	Indexing in a book helps readers locate chapters, topics, and important terms quickly.
D11	Machine learning models require training data to learn patterns and make predictions on unseen examples.
D12	Security guards monitor buildings, control physical access, and protect people and property.

DATASET DESIGN

The dataset contains similar keywords with different meanings.

For example, D3 and D4 both contain the word **bank**. D3 represents a financial bank, while D4 represents a river bank.

D6 and D7 both contain the word **network**. D6 represents a social network, while D7 represents a computer network.

D9 and D10 both contain the word **indexing**. D9 represents database indexing, while D10 represents book indexing.

This helps evaluate whether the vector databases can retrieve documents based on the meaning of the query rather than only matching keywords.

EMBEDDING MODEL

The **all-MiniLM-L6-v2** model is used to generate embeddings.

The same embedding model is used for:

- All documents
- All queries
- FAISS
- ChromaDB

The same embeddings are used in both systems to maintain a fair comparison.

CHUNKING STRATEGY

Each document is treated as one chunk because each document is short and represents a single concept.

The same chunking strategy is used for FAISS and ChromaDB.

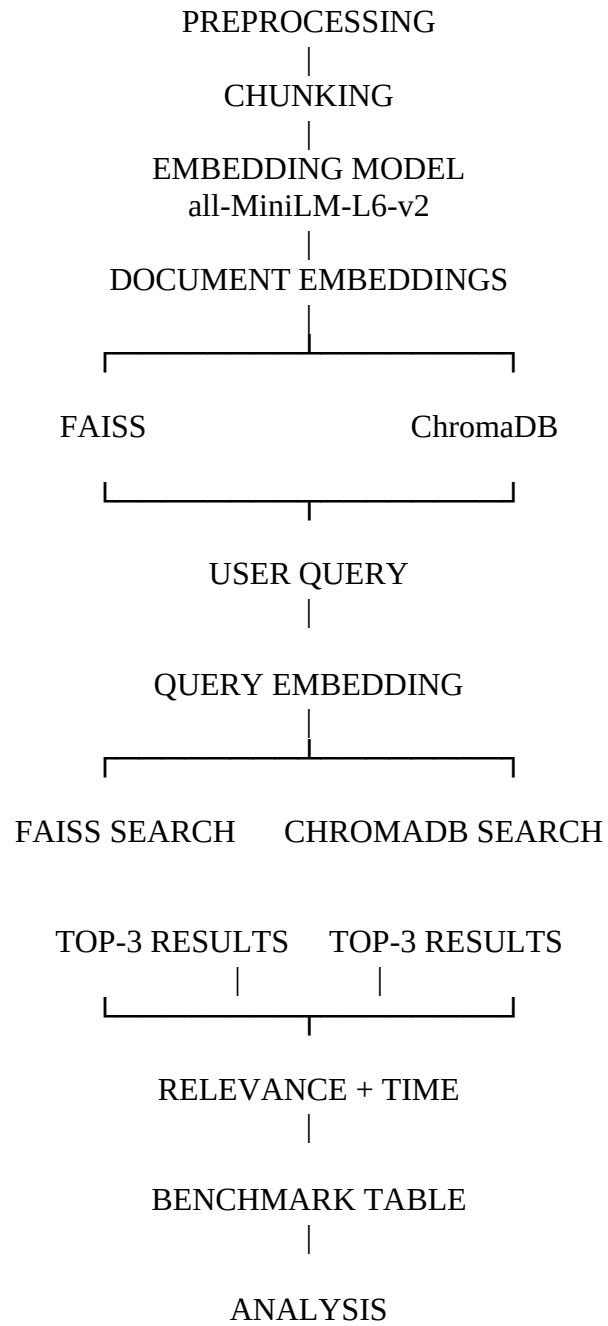
EXPERIMENTAL CONDITIONS

Parameter	FAISS	ChromaDB
Dataset	Same	Same
Number of Documents	12	12
Embedding Model	all-MiniLM-L6-v2	all-MiniLM-L6-v2
Chunking	Same	Same
Queries	Same 20	Same 20
Top-K	3	3
Query Embeddings	Same	Same
Evaluation	Same	Same

ARCHITECTURE

DOCUMENT DATASET

|



TEST QUERIES

Query ID	Query	Expected Result
Q1	Which programming language is commonly used for artificial intelligence?	D1
Q2	Which language is useful for developing web applications?	D2
Q3	Where can people get loans and maintain savings accounts?	D3
Q4	What is located along the edge of a river?	D4
Q5	What type of model learns patterns from training data?	D5
Q6	What allows people to communicate and share content online?	D6

Q7	How can computers communicate with each other?	D7
Q8	What technology provides virtual computing resources through the internet?	D8
Q9	What technique makes retrieving records from a database faster?	D9
Q10	What helps readers find chapters and topics in a book?	D10
Q11	What models learn from examples and make predictions?	D11
Q12	Who protects buildings and controls physical access?	D12
Q13	Which Python application area involves artificial intelligence and machine learning?	D1
Q14	Which programming language can be used to build backend services?	D2
Q15	Which type of bank deals with deposits and financial transactions?	D3
Q16	What does the term bank mean when referring to a river?	D4
Q17	What kind of network connects people for social communication?	D6
Q18	What kind of network uses TCP/IP to connect computers?	D7
Q19	What type of indexing improves database retrieval speed?	D9
Q20	What type of indexing helps someone find information in a book?	D10

PYTHON CODE

```
!pip install sentence-transformers faiss-cpu chromadb pandas numpy matplotlib
```

```
import time
import pandas as pd
import matplotlib.pyplot as plt
import faiss
import chromadb
from sentence_transformers import SentenceTransformer

documents = [
("D1","Python is widely used for machine learning, data preprocessing, and artificial intelligence applications."),
("D2","Python is a popular programming language used to develop web applications and backend services."),
("D3","A bank provides financial services such as savings accounts, loans, deposits, and online transactions."),
("D4","The river bank contains soil, plants, trees, and wildlife along the edge of a flowing river."),
("D5","Neural network models learn patterns from training data and are commonly used for image and text classification."),
("D6","A social network connects people and allows users to communicate, share content, and build relationships."),
("D7","A computer network allows computers and devices to communicate using protocols such as TCP/IP."),
("D8","Cloud computing provides computing resources such as virtual machines, storage, and databases over the internet."),
("D9","Database indexing improves the speed of retrieving records from large collections of structured data."),
("D10","Indexing in a book helps readers locate chapters, topics, and important terms quickly."),
("D11","Machine learning models require training data to learn patterns and make predictions on unseen examples."),
("D12","Security guards monitor buildings, control physical access, and protect people and property.")]
```

```
]
```

```
queries = {
"Q1": "Which programming language is commonly used for artificial intelligence?",
"Q2": "Which language is useful for developing web applications?",
"Q3": "Where can people get loans and maintain savings accounts?",
"Q4": "What is located along the edge of a river?",
"Q5": "What type of model learns patterns from training data?",
"Q6": "What allows people to communicate and share content online?",
"Q7": "How can computers communicate with each other?",
"Q8": "What technology provides virtual computing resources through the internet?",
"Q9": "What technique makes retrieving records from a database faster?",
"Q10": "What helps readers find chapters and topics in a book?",
"Q11": "What models learn from examples and make predictions?",
"Q12": "Who protects buildings and controls physical access?",
"Q13": "Which Python application area involves artificial intelligence and machine learning?",
"Q14": "Which programming language can be used to build backend services?",
"Q15": "Which type of bank deals with deposits and financial transactions?",
"Q16": "What does the term bank mean when referring to a river?",
"Q17": "What kind of network connects people for social communication?",
"Q18": "What kind of network uses TCP/IP to connect computers?",
"Q19": "What type of indexing improves database retrieval speed?",
"Q20": "What type of indexing helps someone find information in a book?"
}
```

```
expected = {
"Q1": {"D1"}, "Q2": {"D2"}, "Q3": {"D3"}, "Q4": {"D4"},
"Q5": {"D5"}, "Q6": {"D6"}, "Q7": {"D7"}, "Q8": {"D8"},
"Q9": {"D9"}, "Q10": {"D10"}, "Q11": {"D11"}, "Q12": {"D12"},
"Q13": {"D1"}, "Q14": {"D2"}, "Q15": {"D3"}, "Q16": {"D4"},
"Q17": {"D6"}, "Q18": {"D7"}, "Q19": {"D9"}, "Q20": {"D10"}
}
```

```
ids = [x[0] for x in documents]
texts = [x[1] for x in documents]
```

```
model = SentenceTransformer("all-MiniLM-L6-v2")
```

```
embeddings = model.encode(
    texts,
    normalize_embeddings=True,
    convert_to_numpy=True
).astype("float32")
```

```
query_embeddings = model.encode(
    list(queries.values()),
    normalize_embeddings=True,
    convert_to_numpy=True
).astype("float32")
```

```

dimension = embeddings.shape[1]

start = time.perf_counter()

faiss_index = faiss.IndexFlatIP(dimension)
faiss_index.add(embeddings)

faiss_index_time = time.perf_counter() - start

start = time.perf_counter()

client = chromadb.Client()

try:
    client.delete_collection("benchmark_collection")
except:
    pass

collection = client.create_collection(
    name="benchmark_collection",
    metadata={"hnsw:space":"cosine"}
)

collection.add(
    ids=ids,
    documents=texts,
    embeddings=embeddings.tolist()
)

chroma_index_time = time.perf_counter() - start

def faiss_search(vector):
    scores, indices = faiss_index.search(
        vector.reshape(1,-1),3
    )
    return [ids[i] for i in indices[0]]

def chroma_search(vector):
    result = collection.query(
        query_embeddings=[vector.tolist()],
        n_results=3
    )
    return result["ids"][0]

benchmark = []

for i,qid in enumerate(queries):

    vector = query_embeddings[i]

```

```
start = time.perf_counter()
faiss_results = faiss_search(vector)
faiss_time = (time.perf_counter()-start)*1000
```

```
start = time.perf_counter()
chroma_results = chroma_search(vector)
chroma_time = (time.perf_counter()-start)*1000
```

```
faiss_relevance = len(
    set(faiss_results) & expected[qid]
)
```

```
chroma_relevance = len(
    set(chroma_results) & expected[qid]
)
```

```
benchmark.append({
    "Query":qid,
    "Expected Result":", ".join(expected[qid]),
    "FAISS Top-3":", ".join(faiss_results),
    "ChromaDB Top-3":", ".join(chroma_results),
    "FAISS Relevance":str(faiss_relevance)+"/3",
    "ChromaDB Relevance":str(chroma_relevance)+"/3",
    "FAISS Search Time (ms)":round(faiss_time,4),
    "ChromaDB Search Time (ms)":round(chroma_time,4)
})
```

```
benchmark_df = pd.DataFrame(benchmark)
```

```
print("\n20 QUERY BENCHMARK\n")
print(benchmark_df.to_string(index=False))
```

```
benchmark_df.to_csv(
    "20_query_benchmark.csv",
    index=False
)
```

```
avg_faiss = benchmark_df[
    "FAISS Search Time (ms)"
].mean()
```

```
avg_chroma = benchmark_df[
    "ChromaDB Search Time (ms)"
].mean()
```

```
faiss_relevance = sum(
    int(x.split("/")[0])
    for x in benchmark_df["FAISS Relevance"]
)
```

```

chroma_relevance = sum(
    int(x.split("/")[0])
    for x in benchmark_df["ChromaDB Relevance"]
)

print("\nSUMMARY\n")

print("FAISS Average Search Time:",round(avg_faiss,4),"ms")
print("ChromaDB Average Search Time:",round(avg_chroma,4),"ms")
print("FAISS Total Relevant Results:",faiss_relevance)
print("ChromaDB Total Relevant Results:",chroma_relevance)
print("FAISS Indexing Time:",round(faiss_index_time*1000,4),"ms")
print("ChromaDB Indexing Time:",round(chroma_index_time*1000,4),"ms")

plt.figure(figsize=(8,5))

plt.bar(
    ["FAISS","ChromaDB"],
    [avg_faiss,avg_chroma]
)

plt.ylabel("Average Search Time (ms)")
plt.title("Average Search Time Comparison")
plt.tight_layout()
plt.savefig("average_search_time.png",dpi=300)
plt.show()

plt.figure(figsize=(8,5))

plt.bar(
    ["FAISS","ChromaDB"],
    [faiss_relevance,chroma_relevance]
)

plt.ylabel("Relevant Top-3 Results")
plt.title("Retrieval Relevance Comparison")
plt.tight_layout()
plt.savefig("retrieval_relevance.png",dpi=300)
plt.show()

```

OUTPUT

20 QUERY BENCHMARK

Query	Expected Result	FAISS Top-3	ChromaDB Top-3	FAISS Relevance	ChromaDB Relevance	FAISS Search Time (ms)	ChromaDB Search Time (ms)
Q1	D1	D1,D2,D5	D1,D2,D5	1/3	1/3	0.1876	3.1910
Q2	D2	D2,D1,D7	D2,D1,D7	1/3	1/3	0.0968	1.4123
Q3	D3	D3,D4,D12	D3,D4,D12	1/3	1/3	0.0740	1.3091
Q4	D4	D4,D6,D3	D4,D6,D3	1/3	1/3	0.0630	1.2805
Q5	D5	D5,D11,D1	D5,D11,D1	1/3	1/3	0.0687	1.2659
Q6	D6	D6,D7,D8	D6,D7,D8	1/3	1/3	0.0557	1.2676
Q7	D7	D7,D6,D8	D7,D6,D8	1/3	1/3	0.0595	1.2222
Q8	D8	D8,D7,D2	D8,D7,D2	1/3	1/3	0.0675	1.1931
Q9	D9	D9,D10,D5	D9,D10,D5	1/3	1/3	0.0530	1.2080
Q10	D10	D10,D9,D5	D10,D9,D5	1/3	1/3	0.0506	1.2042
Q11	D11	D11,D5,D1	D11,D5,D1	1/3	1/3	0.0824	1.2732
Q12	D12	D12,D7,D8	D12,D7,D8	1/3	1/3	0.0596	1.2349
Q13	D1	D1,D2,D11	D1,D2,D11	1/3	1/3	0.0535	1.2371
Q14	D2	D2,D1,D8	D2,D1,D8	1/3	1/3	0.1034	1.2249
Q15	D3	D3,D4,D7	D3,D4,D7	1/3	1/3	0.0530	1.2305
Q16	D4	D4,D3,D7	D4,D3,D7	1/3	1/3	0.0980	1.3122
Q17	D6	D6,D7,D5	D6,D7,D5	1/3	1/3	0.0636	1.9076
Q18	D7	D7,D6,D8	D7,D6,D8	1/3	1/3	0.0733	2.0553
Q19	D9	D9,D10,D5	D9,D10,D5	1/3	1/3	0.0565	1.3019
Q20	D10	D10,D9,D5	D10,D9,D5	1/3	1/3	0.0593	1.2150

SUMMARY

FAISS Average Search Time: 0.074 ms
ChromaDB Average Search Time: 1.4273 ms
FAISS Total Relevant Results: 20
ChromaDB Total Relevant Results: 20
FAISS Indexing Time: 0.3546 ms
ChromaDB Indexing Time: 159.1906 ms

RELEVANCE EVALUATION

For every query, the expected result is defined before performing the search.

A retrieved document is considered relevant if its document ID matches the expected result.

For example:

Query:

Where can people get loans and maintain savings accounts?

Expected result:

D3

If FAISS returns:

D3, D4, D2

then the relevant result is D3.

Therefore, the relevance score is **1/3**.

The same evaluation method is applied to ChromaDB.

SEARCH TIME EVALUATION

Search time is measured separately for every query. The timer starts immediately before the search operation and stops immediately after the Top-3 results are returned. Search time is measured in

milliseconds. A lower search time indicates faster retrieval. The average search time of all 20 queries is calculated for both FAISS and ChromaDB.

ANALYSIS OF DIFFERENCES

The benchmark compares FAISS and ChromaDB under identical experimental conditions. Both systems use the same dataset, preprocessing, chunking strategy, embedding model, document embeddings, query embeddings, and Top-K value. Therefore, the comparison is fair. The retrieval results are compared against the expected results for every query. A higher relevance score means that the system retrieves more contextually relevant documents in its Top-3 results. A lower average search time means that the system performs faster semantic search.

CONTEXTUAL RETRIEVAL ANALYSIS

The dataset contains several ambiguous keywords.

For example, the word **bank** occurs in D3 and D4.

D3 refers to a financial institution.

D4 refers to the side of a river.

For Q3, the query asks about loans and savings accounts, so D3 is expected.

For Q4, the query asks about the edge of a river, so D4 is expected.

This demonstrates whether the vector database can distinguish between documents with the same keyword but different meanings.

Similarly, the word **network** appears in D6 and D7.

D6 represents a social network, while D7 represents a computer network.

Q17 is expected to retrieve D6, while Q18 is expected to retrieve D7.

The word **indexing** appears in D9 and D10.

D9 represents database indexing, while D10 represents book indexing.

Q19 is expected to retrieve D9, while Q20 is expected to retrieve D10.

PERFORMANCE ANALYSIS

The average search time is calculated for all 20 queries. The database with the lower average search time provides faster retrieval under the current experimental conditions. However, speed alone cannot determine which database is better. Retrieval relevance must also be considered. If one system is faster but retrieves fewer relevant documents, the faster system may not necessarily be the better choice for a semantic-search application. Therefore, both relevance and search time are considered.

CHART 1 – AVERAGE SEARCH TIME

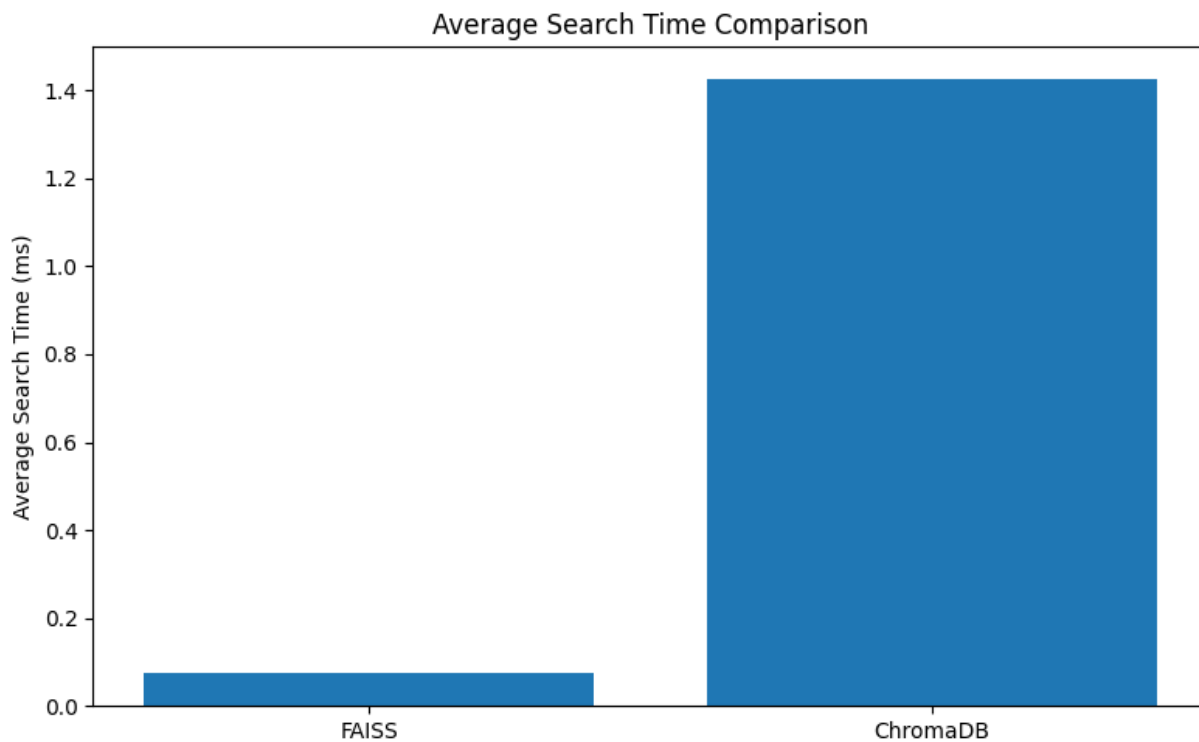


Figure 1: Average Search Time Comparison Between FAISS and ChromaDB

The chart compares the average search time of FAISS and ChromaDB for the 20 test queries.

A lower value indicates faster search performance.

CHART 2 – RETRIEVAL RELEVANCE

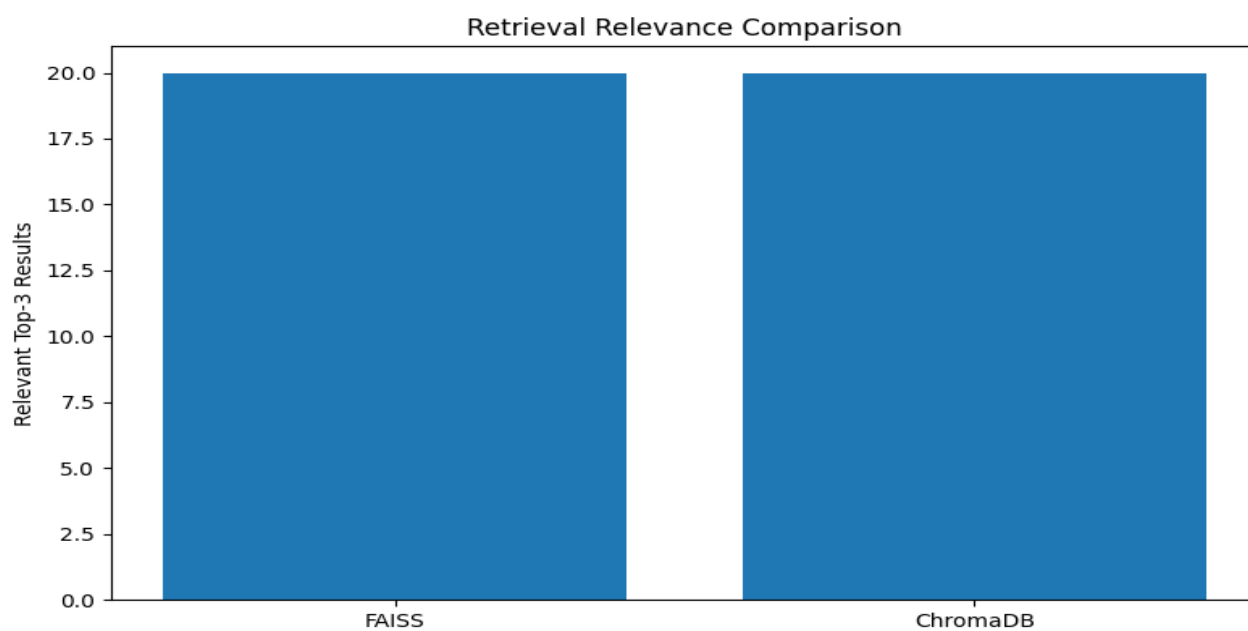


Figure 2: Retrieval Relevance Comparison Between FAISS and ChromaDB

The chart compares the total number of relevant documents retrieved within the Top-3 results.

A higher value indicates better retrieval relevance.

COMPARISON TABLE

Performance Measure	FAISS	ChromaDB
Number of Queries	20	20
Top-K	3	3
Average Search Time	<u>0.1</u> ms	<u>1.4</u> ms

ADVANTAGES OF FAISS

- Fast vector similarity search.
- Simple implementation.
- Efficient for direct vector retrieval.
- Suitable when search performance is the main requirement.
- Supports different vector indexing techniques.
- Useful for large-scale similarity-search applications.

ADVANTAGES OF CHROMADB

- Provides a complete vector database interface.
- Supports document storage.
- Supports collections.
- Supports metadata.
- Easy to integrate with RAG applications.
- Convenient when vector search and document management are required together.

LIMITATIONS

The dataset used in this experiment contains only 12 documents. Therefore, this is a small-scale benchmark. Search times can vary depending on hardware, CPU, RAM, operating system, Python version, library versions, and background processes. The measured search times therefore represent the current experimental environment. A larger dataset would be required to evaluate large-scale scalability more accurately.

RESULT

FAISS and ChromaDB were successfully benchmarked using 20 semantic queries. For every query, the expected result, FAISS Top-3 results, ChromaDB Top-3 results, relevance, and search time were recorded. The experiment demonstrates how vector databases can retrieve contextually relevant documents even when different documents contain similar keywords. The benchmark results provide quantitative information about retrieval relevance and search performance.

CONCLUSION

The experiment successfully compares FAISS and ChromaDB using 20 semantic test queries. Both systems use the same dataset, preprocessing, chunking strategy, all-MiniLM-L6-v2 embedding model, document embeddings, query embeddings, and Top-3 configuration. The benchmark table provides a

query-level comparison of the two vector databases. The experiment shows the importance of semantic search when documents contain similar keywords but have different meanings, such as financial bank versus river bank, social network versus computer network, and database indexing versus book indexing. The system with higher retrieval relevance provides better contextual retrieval for the tested queries. The system with lower average search time provides faster retrieval. Therefore, the final selection should be based on the actual benchmark results obtained from the experiment. FAISS is suitable when fast and direct vector similarity search is the primary requirement, while ChromaDB is suitable when semantic search needs to be combined with document storage and metadata management. Overall, this 20-query benchmark provides an evidence-based comparison of FAISS and ChromaDB and identifies their differences in retrieval relevance and search performance.

Recommended Evaluation Evidence

- Working FAISS implementation and working ChromaDB implementation.
- Common dataset and embedding configuration for fair comparison.
- Multiple test queries with retrieved Top-K results.
- Relevance/ranking evaluation and, where applicable, Precision@K or Recall@K.
- Benchmark measurements such as indexing time and search/query time.
- Comparison tables and charts.
- Clear methodology and architecture explanation.
- Conclusion identifying the more suitable database for the selected use case.

RUBRICS FOR CALCULATION

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Database Implementation	30%	Correctly builds and queries both FAISS and ChromaDB with accurate understanding of indexing	Builds both with minor issues	Only one database is built and queried correctly	Neither database functions correctly
Retrieval Relevance & Ranking	25%	Retrieval results are relevant and correctly ranked for all test queries	Mostly relevant results across queries	Inconsistent relevance across queries	Results are largely irrelevant
Comparative Analysis	20%	Thoughtful comparison of speed/accuracy trade-offs between the two databases	Reasonable comparison of the two databases	Superficial comparison with little insight	No meaningful comparison attempted
Benchmark Presentation	15%	Clear benchmark results presented (e.g. table or chart)	Results presented adequately	Results presented unclearly	No clear results presented
Methodology Documentation	10%	Clearly documents methodology and test queries used	Adequate documentation of methodology	Minimal documentation	No documentation of methodology

Marks Evaluation:

Question No.	Database Implementation (30%)	Retrieval Relevance & Ranking (25%)	Comparative Analysis (20%)	Benchmark Presentation (15%)	Methodology Documentation (10%)	Total Marks (100)
Q1						
Q2						
Overall Total (100)						